The Superior University, Lahore

LAB-TASK (7) (Spring-2026)

Course Title:	Programming for AI (Lab)				Course Code:	-	Credit Hours	1
Instructor:	Sir Rasikh				Program:	BSDS		
Semester:	4th	Batch:	Morning	Section:	4A	Date:	-	
Time Allowed:	-				Maximum Marks:		N/A	
Student's Name:	Shumaila Maryam				Roll No.	SU92-BSDSM-S25-062		

<u>Work Assigned:</u>

Question # 01:

Task 1; Create any of the following, can use Free API & develop its "Flask" based back-end only:

1. **Cooking / Recipe - App**

DEPARTMENT OF SOFTWARE ENGINEERING

Recipe Discovery Web Application

Introduction

The Recipe Discovery Web Application is a comprehensive Flask-based system designed to help users discover, explore, and manage meal recipes from around the world. This project demonstrates advanced web development concepts including RESTful API integration, dynamic routing, data processing, and responsive template rendering. Built using Python Flask with the requests library, the system seamlessly integrates with TheMealDB API to provide access to thousands of recipes with detailed ingredients, instructions, and nutritional categories.

The application comprises 8 distinct routes handling various functionalities from basic recipe search to advanced category filtering and shopping list management. Users can search recipes by name, browse alphabetically through over 26 letter categories, explore 14+ meal categories, and view comprehensive details including ingredients with measurements, preparation times, and cooking instructions.

Why I Created This

The primary objective of this project is to demonstrate and strengthen full-stack web development skills across frontend and backend technologies. Through this implementation, I aimed to:

- API Integration Mastery: Learn to consume external REST APIs, handle JSON responses, and manage API endpoint configurations for different data retrieval patterns.
- Flask Framework Proficiency: Develop expertise in Flask routing, template rendering, request parameter handling, and application configuration management.
- Data Transformation Skills: Practice extracting structured information from complex nested JSON responses and transforming it into user-friendly formats.
- Dynamic Web Applications: Understand server-side rendering, template inheritance, and dynamic content generation based on user interactions.
- Error Handling Strategies: Implement graceful error handling for API failures, missing data, and invalid user inputs to ensure robust application behavior.
- Feature Engineering: Create intelligent estimation algorithms for missing data like cooking and preparation times based on recipe categories.
- User Experience Design: Build intuitive navigation systems including search, category browsing, and alphabetical filtering for enhanced user engagement.

- Web Application Architecture: Experience the complete workflow from API integration to frontend presentation following MVC architectural patterns.
- This project serves as both an educational exercise and a practical demonstration of building production-ready web applications that solve real user needs in the culinary domain.

How It Works

1. Application Initialization and Configuration

The system begins by importing necessary libraries and initializing the Flask application instance. Five API endpoint constants are configured for different TheMealDB operations including search by name, lookup by ID, filter by letter, category listing, and category filtering.

Application Setup:

```
import requests
from flask import   Flask, render_template, request

app = Flask(__name__)


MEALDB_SEARCH = "https://www.themealdb.com/api/json/v1/1/search.php?s="
MEALDB_ID = "https://www.themealdb.com/api/json/v1/1/lookup.php?i="
MEALDB_LETTER = "https://www.themealdb.com/api/json/v1/1/search.php?f="
MEALDB_CATEGORIES = "https://www.themealdb.com/api/json/v1/1/list.php?c=list"
MEALDB_CATEGORY = "https://www.themealdb.com/api/json/v1/1/filter.php?c="
```

2. Home Page Route

The home route fetches featured recipes from the first three letters of the alphabet to display diverse meal options. Each letter retrieves meals from the API, selecting the first 2 meals per letter for a total of 6 featured recipes on the homepage.

Home Route Implementation:

```
@app.route('/')
def home():
    try:
        featured = []
        for letter in ['a', 'b', 'c']:
            resp = requests.get(MEALDB_LETTER + letter)
            data = resp.json()
            if data['meals']:
                featured.extend(data['meals'][:2])
        return render_template('index.html', featured=featured)
    except:
        return render_template('index.html', featured=[])
```

3. Recipe Search Functionality

Users can search for recipes by entering food names through the search interface. The search route retrieves the query parameter, makes an API call, and returns matching recipes or appropriate error messages when no results are found.

Search Route:

```python
@app.route('/search')
def search():
    """Search for recipes by name"""
    food = request.args.get('food')
    if not food:
        return render_template('search.html', meals=None)

    try:
        resp = requests.get(MEALDB_SEARCH + food)
        data = resp.json()

        if data['meals'] is None:
            return render_template('search.html', meals=None, error=f"No recipes

        return render_template('search.html', meals=data['meals'], food=food)
    except Exception as e:
        return render_template('search.html', meals=None, error=str(e))
```

4. Ingredient Extraction System

The system includes a specialized function to extract ingredients and measurements from meal data. TheMealDB API returns ingredients in 20 separate fields requiring intelligent parsing to combine ingredients with their corresponding measurements into structured dictionaries.

Ingredient Extraction Function:

```python
def extract_ingredients(meal_data):
    """Extract ingredients and measures from meal data"""
    ingredients = []
    for i in range(1, 21):
        ing_key = f'strIngredient{i}'
        measure_key = f'strMeasure{i}'

        if ing_key in meal_data and meal_data[ing_key]:
            ingredient = meal_data[ing_key].strip()
            measure = meal_data[measure_key].strip() if measure_key in meal_data

            if ingredient:
                ingredients.append({
                    'ingredient': ingredient,
                    'measure': measure
                })
    return ingredients
```

5. Cooking Time Estimation

Since TheMealDB API does not provide preparation and cooking times, the application implements an intelligent estimation system based on recipe categories. Different food categories have pre-configured time ranges reflecting typical cooking requirements.

6. Individual Meal Details

The meal detail route fetches comprehensive information for a specific recipe using its unique ID. This includes full instructions, ingredient lists, preparation times, cooking times, category information, and meal images.

Application Runner:

```
if __name__ == "__main__":
    app.run(host='localhost', port=8080, debug=True)
```

Summary

1. Comprehensive Web Solution: The Recipe Discovery Web Application implements a complete web development pipeline from API integration through frontend presentation, demonstrating end-to-end full-stack capabilities.

2. Feature-Rich Functionality: The system provides 8 distinct routes supporting multiple user interaction patterns including search, browsing, category filtering, and shopping list management for enhanced user experience.

3. Production-Ready Architecture: The application includes proper error handling, API response validation, data transformation utilities, and modular function design making it suitable for deployment in real-world scenarios.

4. Flask Excellence: Leverages Flask's powerful routing system, template engine, and request handling mechanisms to create a responsive and maintainable web application architecture.

5. Data Processing Intelligence: Implements sophisticated data extraction algorithms for ingredients and intelligent estimation systems for missing information like preparation times.

6. Educational Value: Project demonstrates mastery of Python web development, RESTful API consumption, server-side rendering, and modern web application design patterns.

Technologies Used

Technology	Purpose	Version
Python	Backend programming language	3.7+
Flask	Web framework	≥2.0.0
Requests	HTTP library for API calls	≥2.25.0
Jinja2	Template engine (Flask built-in)	≥3.0.0
TheMealDB API	Recipe data source	v1
HTML5	Frontend structure	Standard
CSS3	Styling and layout	Standard

Output and Results

Application Features

Feature	Status
Recipe Search by Name	Functional
Browse by Alphabet (A-Z)	Functional
Category Listing	Functional
Category Filtering	Functional
Detailed Meal Information	Functional
Ingredient Extraction	Functional
Cooking Time Estimation	Functional
Shopping List Management	Functional

Route Information

Route Path	HTTP Method	Purpose
/	GET	Home page with featured recipes
/search	GET	Search recipes by name
/meal/<meal_id>	GET	Display detailed meal information
/letter/<letter>	GET	Browse recipes by first letter
/categories	GET	List all recipe categories
/category/<name>	GET	Show meals in specific category
/shopping-list	GET	Manage shopping list